

INF-1400: Object-oriented programming

Mandatory assignment 1

Jens Christian Valen Leynse

February 13th, 2017

1 Introduction

The assignment was to create a simple clone of the classic game Breakout. The gamecode is written in python 3 using the pygame library. A goal of the assignment is implementing the game using classes and methods.

1.1 Requirements

1. Implement the game in accordance with object-oriented design, use objects and classes.
2. The platform should be controlled by the mouse or keyboard.
3. The ball should bounce in a different direction based on where on the platform it hits.
4. A brick disappears when the ball hits it.
5. The ball bounces off the wall and ceiling, angle in = angle out.
6. The game is won when all bricks are removed, the game is lost when the ball hits the bottom of the screen.
7. Well structured and commented code.

2 Technical Background

The code is utilizing import commands to collect overall data or specific data from different files or libraries, making sure to only include relevant files. The Pygame library is used to have an easy access to premade functions. As the assignment required the class system is used on the different object added, to easily define different properties, as well as assigning relevant functions as a part of the given class. The files included like sound and images are added to a library connected to the game folder to have an easy access.

3 Design & Implementation

The code is separated into three files. The resource file which include the 3 functions from the precode to change two variables into a tuple, or to check if two objects are intersecting.

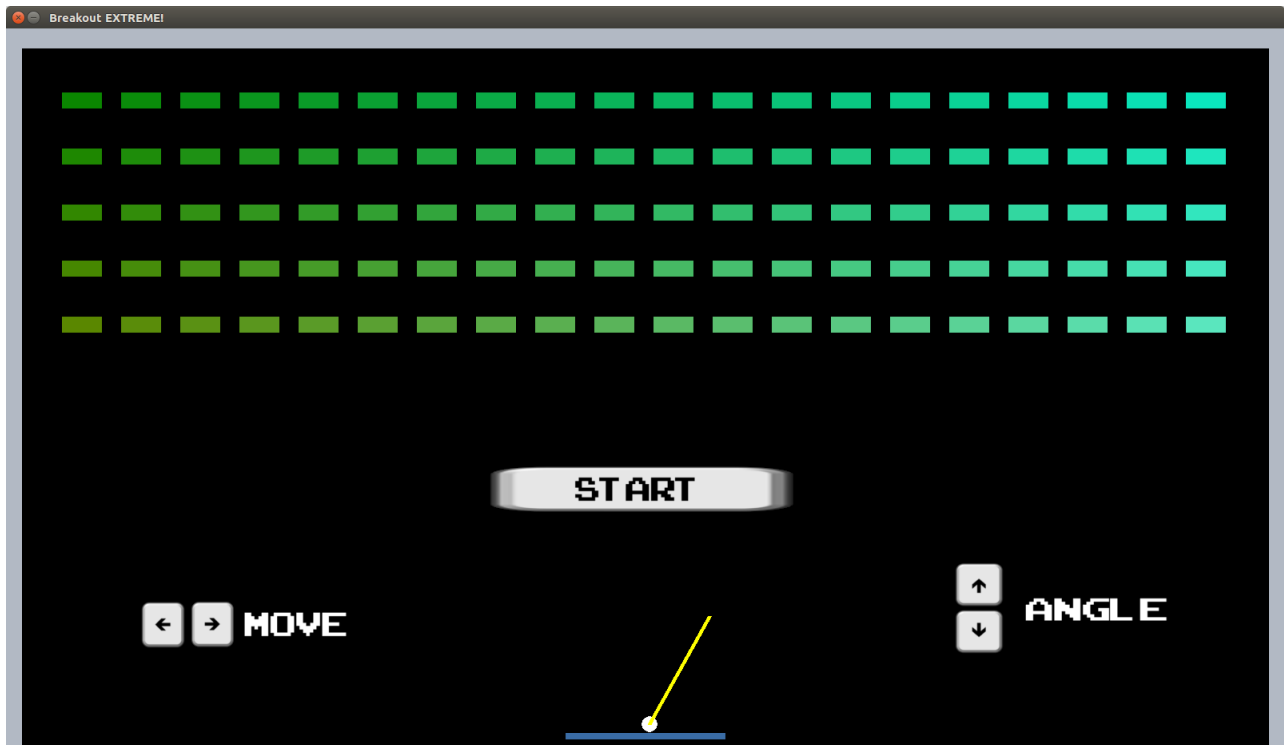
The objects file sets all the values for the created objects given the class of the object.

All of the classes are initialized with their own properties with relevant data values. The objects that needs an ability to move are given so by a function within the class, together with the location data. And the stationary objects are simply given a position.

- **Paddle:** is defined through 4 functions. It is first initialized with the different values that are needed to determine the object. Then the draw function is called from the pygame library to use the values given to display the object on the later determined screen. The Move function uses the moment a certain key is pressed to determine the x position of the paddle and how much it is to change in a direction. The last function determines the angle of which the ball is to bounce after a collision with the paddle according to the position on the paddle it hits.
- **Ball:** is defined through 5 functions. Like the paddle it is defined though an initializer and the same draw definition. The speed vector of the ball is determined through a vector from the balls current position, with the balls move function. The next function which changes the ball direction changes the speed vector from a positive value to a negative and reversed. The last function is just a feature added, so not a part of the original assignment, it displays the vector of the ball.
- **Brick:** Much like the paddle and ball this object is initialized and drawn using different functions. But since this object is to be stationary the only other definition it gets is one to change the direction of the ball if the object is hit.
- **Walls:** Simply draws the walls of the game given the screen height and width.
- **Align:** Assigns positional names to easily place images to different parts of the screen.
- **Pictures:** First it stores the set images in a library with an easy access name. Then it checks the position given from the previous object align. Lastly it fetches the image, transforms the image if the scale is different and displays the image on the screen.

The last and most important file is the game file. Here the first thing that is set is a game mode position to easily jump between the different game scenarios. In the main game function, the important values are set. Either as variables later accessed or as a name given to shorten the call to an object. The mode, music is also set here together with a created list.

The part that runs the game is collected into the games while loop.



The game loop goes through several objectives before it restarts. In order these are; make the screen and add a clock. Play the game if gamemode is set to play, with an easy exit option. A game command to start. All of the object functions that the program has to draw, or add or change the objects values. The list is the opponent objects at the start of each round, when one gets hit, it gets removed. The start angle of the ball and the mathematical functions to move the angle along a circle motion is set.

Lastly four if statements determines the games state and mode. If the game state is inactive, an introduction image is displayed on the screen and the ball can change its start angle. Once the spacebar key is pressed, the game state will change to active, the vector and introduction image will disappear, and the ball and paddle will be able to move. The games state determines what mode the game is to change into after one of the statements becomes true. If the ball exits the screen the game mode is changed to game over, and if the brick objects in the list is removed, the game mode is changed into the you win mode.



The different endgame scenarios both display a different image to take over the screen, saying the results of the playthrough and giving the player the opportunity to retry changing the gamemode back to play. Here the states of the gameplay together with the list of the brick objects gets reset. The player can also choose to exit the game by pressing "escape".

5 Discussion

The code was first written only in the game file written over the example from the precode. All of the objects except for the bricks was placed in before I got informed of how I was supposed to implement the class system to define all the objects. The conversion went by quite smoothly with just a simple blunder that took a little time to figure out (the positional values given to the paddle was inside the loop).

The feature to be able to change the starting angle of the ball was thought up early in the process, but adding mathematical functions to keep the angle and the speed in a pixel perfect half-circle proved difficult. Several of the values given to change the angle is found through experimentation and not through any equation. Several versions of the list implementation got tested, but only the final version was simple enough, yet gives the opportunity to have an easy color scale along the objects.

The idea of setting the different scenarios in the game loop into different "modes", was given to me by a friend at one of the later parts of the code writing. It was however relatively easy to set up and provided an easy opportunity to jump about in the loop, without having every value set each time for it to be removed further down in the code.

The endgame images were first implemented as separate objects, because of my uncertainty of how to change object values differently for different images within one object. With some help the idea of having an object to determine the screen position, and another object to include a library where the images could be added was created. The other images could then be added by just adding the images file position in the game folder. Lastly the features of sound and music got added as a bonus since implementing them in the code proved easy with the functions given from the pygame library.

6 Conclusion

A working breakout clone was created and although it doesn't include several of the originals extra features, it works as intended given the requirements. The goal to get a better understanding of python and thereby taking use of the class system and other object specified functions was also met.

References

The pygame library with its many functions included
<https://pygame.org/docs/>